
Technical Documentation

VulnMiner

Author: GUITARD Paul

Date: october, 2026

Contents

1	Project Context and Problem Statement	2
1.1	Software Vulnerability Detection	2
1.2	LLMs and Repository Context	3
1.3	Vulnerability Dataset Generation	4
2	Technical Implementation	5
2.1	Operating System, Languages and Dependencies	5
2.2	Pipeline Step 1/3: Vulnerability and Repository Retrieval	6
2.3	Pipeline Step 2/3: GitHub Commit Processing	7
2.4	Pipeline Step 3/3: Commit Parsing and Dataset Entry Generation	9
2.5	Context Extraction	9
2.6	Configuration	10
2.7	Pipeline Architecture	11
2.8	Results	11
2.9	Validation	13
2.10	Known Limitations	13

1 Project Context and Problem Statement

1.1 Software Vulnerability Detection

As software systems continue to increase in size and complexity, manually identifying vulnerabilities becomes increasingly challenging. This has motivated the development of automated vulnerability detection techniques.

Traditional approaches rely on static and dynamic analysis. Static analysis examines source code without executing the program. It can analyze large code bases, provide deterministic results, and identify the location of potential vulnerabilities. However, it generally relies on predefined rules or models and may produce false positives or miss vulnerabilities that require runtime behavior or broader context.

Dynamic analysis, on the other hand, observes the behavior of a program during execution. It can detect vulnerabilities that only become apparent at runtime and is less dependent on purely syntactic patterns. However, it requires the program to be executed with suitable inputs, and its effectiveness is limited by test coverage. It can also become expensive for large software projects.

Before the widespread use of Large Language Models, supervised machine learning models were already being used to classify code as vulnerable or non-vulnerable. Some of these approaches are specifically designed for source code and are therefore referred to as code-specific models. They can learn vulnerability patterns from existing data and identify patterns that are difficult to encode manually. However, they require large and high-quality labeled datasets. Their performance can also be affected by dataset bias or data leakage, and their ability to generalize to unseen projects and vulnerabilities may be limited. Their decisions can also be difficult to explain.

More recently, general-purpose Large Language Models (LLMs) have attracted significant interest for vulnerability detection due to their ability to analyze source code using natural language. They can process code together with broader contextual information and incorporate security-related information provided in natural language. Such security-guided approaches can provide the model with additional information about vulnerability types, security rules, or reasoning steps to guide its analysis. In addition, LLMs can produce explanations alongside their predictions. However, LLMs have limitations related to computational cost and the amount of context that can be provided. Their outputs can also be inconsistent, and they may misinterpret code or lack specific security knowledge. Their ability to analyze a vulnerability therefore strongly depends on the information provided as context.

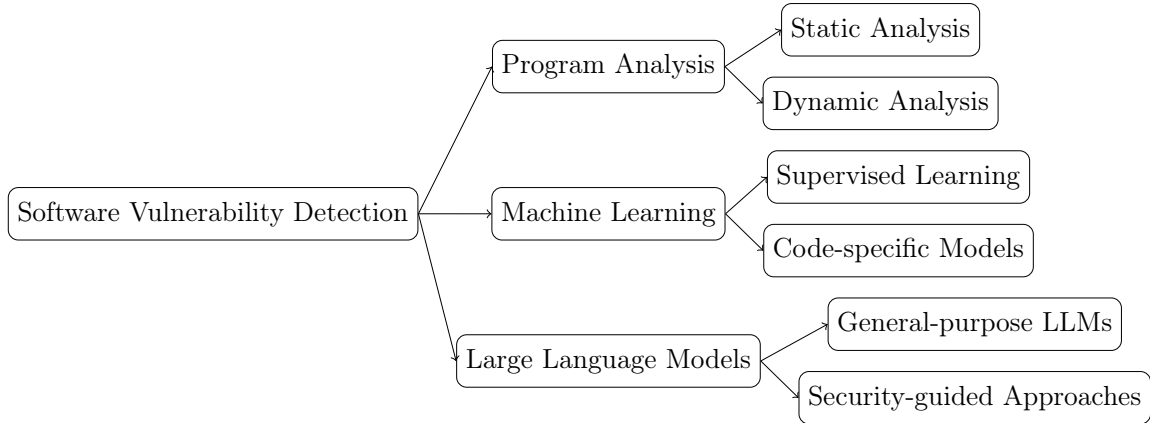


Figure 1: Overview of software vulnerability detection approaches.

1.2 LLMs and Repository Context

As discussed above, each approach has its own advantages and limitations. Despite recent advances, reliable vulnerability detection remains an open challenge. Existing approaches can struggle to generalize beyond the datasets on which they were trained, raising concerns about their effectiveness on real-world software projects.

For LLM-based vulnerability detection, one important area of research is the use of context. In this setting, context refers to the information provided to the model before it generates an answer, such as source code, documentation, or other security-related information. The choice of context can have a significant impact on the model’s ability to analyze a vulnerability.

However, providing more context does not necessarily mean providing better context. Large amounts of information can increase computational and memory costs, while also making it more difficult to identify the information that is actually relevant to the vulnerability. Other constraints, such as confidentiality and the size of the available context window, can also limit the amount of information that can be provided to an LLM.

This makes context selection an important problem for repository-level vulnerability detection. A software repository may contain a large number of files, functions, dependencies, and other pieces of information, while only a small part of them may be relevant to a particular vulnerability. Before an LLM can analyze such a vulnerability, relevant evidence therefore needs to be identified and extracted from repositories. This requires a structured and reproducible way of collecting information from real-world code bases.

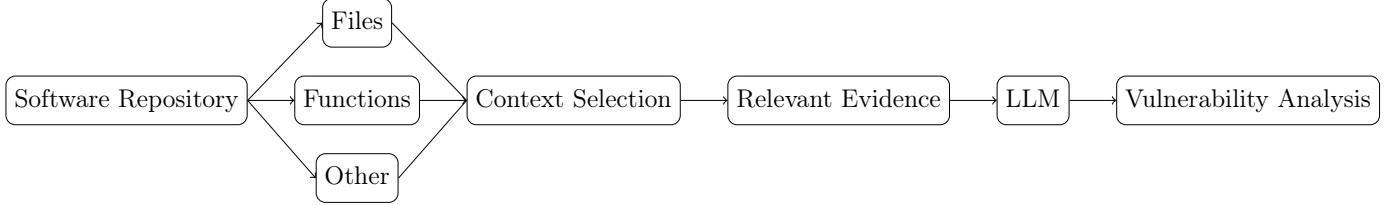


Figure 2: Repository context selection for vulnerability analysis.

1.3 Vulnerability Dataset Generation

The need to collect and structure repository information is also important for the datasets used to train and evaluate vulnerability detection models. These datasets need to be large and diverse, and should ideally be regularly updated with recent vulnerabilities and examples from a wide range of real-world software projects.

This creates a need for reliable and reproducible ways of collecting vulnerability-related data from real-world software repositories. Although many public datasets are already available and widely used in vulnerability detection research, each dataset has its own characteristics and limitations. They may differ in terms of programming languages, time periods, types of vulnerabilities, data quality, false positives, or duplicated samples. Furthermore, public datasets need to be maintained and updated by their respective authors in order to include new vulnerabilities and software projects.

The objective of this work was therefore not to replace existing datasets, but to provide researchers with a tool that can be used to build datasets according to their own requirements. By specifying their own filtering and extraction criteria, researchers can directly retrieve recent vulnerability-related data from public sources and generate datasets in a consistent format, without having to rely entirely on the availability of an external dataset.

This project focuses on this specific problem. The objective was to develop a complete pipeline capable of automatically retrieving vulnerability-related information and source code from GitHub repositories, identifying the relevant changes associated with software vulnerabilities, and generating structured and consistent datasets that can later be used for vulnerability analysis and LLM-based approaches.

2 Technical Implementation

This document describes the technical implementation of the project. The goal is not to detail every line of code, but rather to provide an overview of the pipeline architecture and explain its main components and functionalities as clearly as possible.

2.1 Operating System, Languages and Dependencies

The initial requirements of the project focused on the analysis of software vulnerabilities affecting C and C++ source code. Therefore, the first version of the pipeline was designed to process C and C++ repositories, which are currently the only programming languages supported by the source-code parsing stage.

The implementation required choosing a programming language, as well as several external libraries suited to the needs of the project. C was chosen because it is well suited to low-level processing and provides explicit memory management. It also provides support for parallel programming, such as POSIX threads, which made it a suitable choice for implementing the parallel parts of the pipeline.

The project relies on libraries and system features available on Linux. It is therefore not intended to be used on Windows and was only tested on Linux systems. In addition to the standard libraries and system-provided functionality, the project relies on three third-party libraries:

- **libcurl**: the pipeline needs to retrieve a large amount of data from online services in order to automatically collect the information required to build the datasets. Network communication is therefore an important part of the application, and libcurl provides the HTTP functionality required to communicate with the different APIs.
- **Tree-sitter**: the source files retrieved from repositories need to be analyzed in order to identify and compare the functions affected by a vulnerability. Tree-sitter parses source code into a syntax tree, similar to an Abstract Syntax Tree (AST), which can then be traversed to extract structural information such as functions and their parameters.
- **yyjson**: the APIs used by the pipeline provide data in JSON format, while the generated datasets also use JSON. yyjson is therefore used both to parse the JSON responses received from external services and to generate the JSON files.

The implementation is designed to be as simple as possible to build and use. Particular attention was given to avoiding unnecessary dependencies and keeping the compilation and installation process straightforward.

2.2 Pipeline Step 1/3: Vulnerability and Repository Retrieval

The pipeline can be divided into three main stages. The first stage is responsible for retrieving vulnerability information and identifying GitHub commits related to software vulnerabilities based on a set of predefined criteria. The U.S. National Vulnerability Database (NVD) is used as the main source for this purpose.

The NVD is a regularly updated database that collects information about software vulnerabilities. For each vulnerability, it provides a list of references linking to external resources related to the vulnerability. These references can point to many different types of resources, such as discussion forums, technical documentation, or, most importantly for this project, source-code repositories and commits related to the vulnerability.

The objective of querying the NVD is therefore to retrieve a list of URLs pointing to GitHub commits that correspond to patches for software vulnerabilities. Each NVD entry corresponds to a CVE (Common Vulnerabilities and Exposures), which identifies a specific vulnerability. An entry contains information such as its description, publication date and references to external resources, including GitHub patch URLs. These URLs can then be passed to the second stage of the pipeline, which is responsible for retrieving and processing the corresponding GitHub repositories and commits. To make this possible, the NVD provides an API that can be queried using specified criteria (see the [Configuration](#) section).

```
{
  "cve": {
    "id": "CVE-2023-52267",
    "descriptions": [
      {
        "lang": "en",
        "value": "ehttp 1.0.6 before 17405b9 has a simple_log.cpp out-of-
          bounds-read during error logging for long strings."
      }
    ],
    "references": [
      {
        "url": "https://github.com/hongliuliao/ehttp/commit/17405
          b975948abc216f6a085d2d027ec1cfd5766",
        "tags": ["Patch"]
      },
      {
        "url": "https://github.com/hongliuliao/ehttp/issues/38",
        "tags": ["Exploit"]
      }
    ]
  }
}
```

Figure 3: Simplified example of an NVD JSON response.

Once the data has been successfully retrieved, it is provided in JSON format and is therefore parsed using `yyjson`. Figure 3 shows a simplified example of an NVD entry. Understanding the structure of the NVD response and the different fields that can be returned was necessary in order to extract the information required by the pipeline. However, the complete response and all of its possible fields are not necessary for the purposes of this document. The information required for the next stage is mainly the references associated with each vulnerability, and more specifically the GitHub URLs identified with the “Patch” tag.

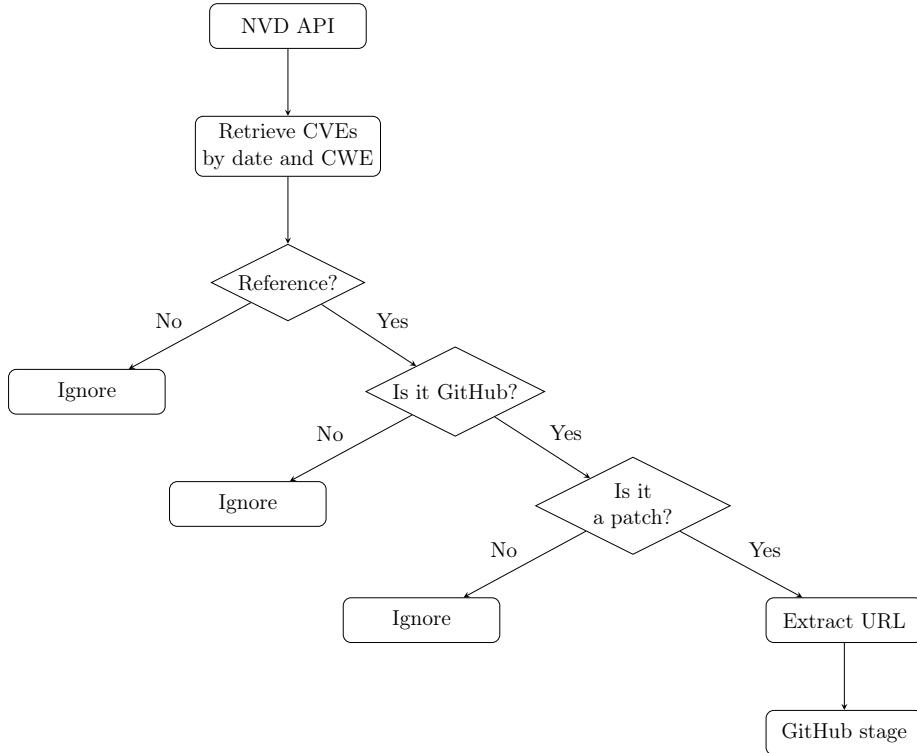


Figure 4: Algorithm used to retrieve GitHub patch references from the NVD.

Regarding network communication, `libcurl` is used to send requests to the API. The details of the network error handling are not discussed further in this document. However, it is important to note that a significant part of the implementation work involved understanding how to communicate reliably with the different APIs used by the pipeline and react appropriately to the different HTTP responses received.

2.3 Pipeline Step 2/3: GitHub Commit Processing

In the previous section, we showed how to obtain a set of GitHub URLs corresponding to vulnerability-related patches. However, additional work is required to analyze each commit and verify whether it satisfies the criteria specified by the user (see the [Configuration](#) section), as well as other mandatory criteria.

To do this, GitHub provides a convenient API. As with the NVD, a significant part of the implementation work consisted in understanding how to communicate with the API and how to interpret the data returned by it.

The first request is used to retrieve the information associated with the commit. This provides, among other things, the parent commit and the list of file paths affected by the commit. The paths can then be analyzed to determine the extensions of the files and to count the total number of affected files.

This allows several invalid commits to be rejected before downloading the actual content of their files. For example, a commit is rejected if it contains files with unsupported extensions or if the number of commit files exceeds the configured limit. Empty commits are also rejected. This early filtering is important because downloading the content of every file can generate a significant amount of network traffic.

If the commit passes these initial checks, the pipeline can optionally retrieve additional context files from the repository (see the [Context](#) section). When context extraction is enabled, a second GitHub API request is made to retrieve the repository tree associated with the commit. This provides the list of files available in the repository without requiring their contents to be downloaded.

The repository tree is then used to identify the files that can be included as context according to the configured context depth. The number of context files is also checked against the configured limits. If there are too many context files or if the total number of affected and context files exceeds the configured limit, the commit is rejected before downloading their contents.

Once all these checks have been completed successfully, the pipeline downloads the contents of the affected files and, when enabled, the selected context files. To retrieve the two versions of an affected file, the pipeline uses the hash of the current commit and the hash of its parent commit. The parent commit therefore provides the “before” version of the files, while the current commit provides the “after” version.

If the content of any required file cannot be retrieved, the commit is rejected. Otherwise, the commit is considered valid and is passed to the next stage of the pipeline, where the source files are parsed and analyzed using Tree-sitter.

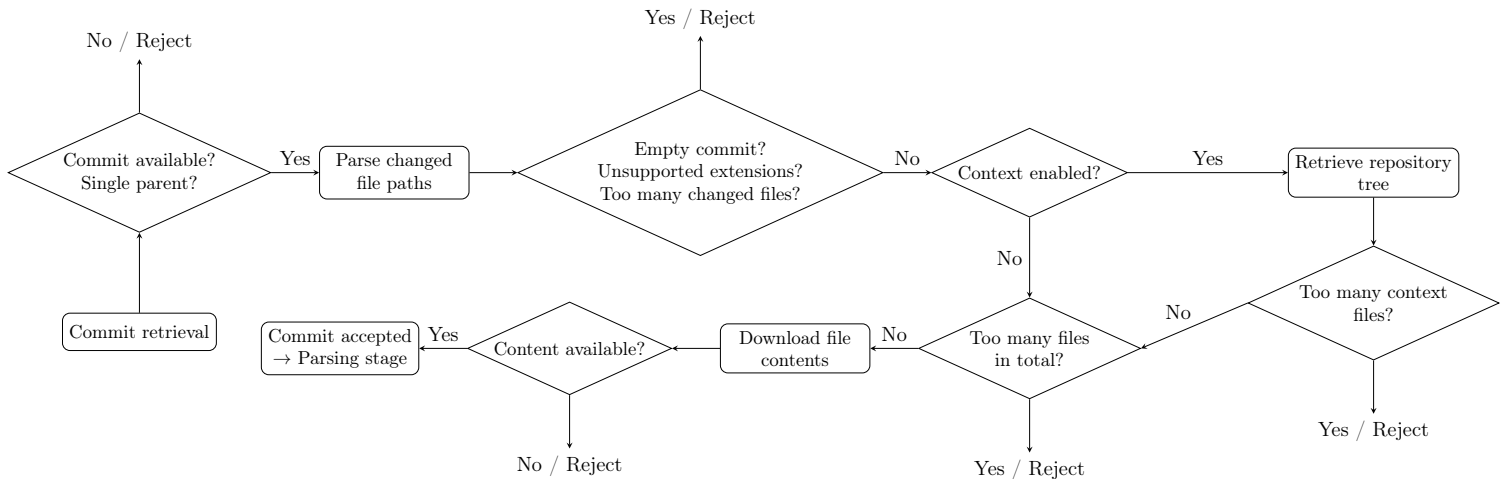


Figure 5: GitHub commit validation and source-code retrieval process.

2.4 Pipeline Step 3/3: Commit Parsing and Dataset Entry Generation

At this stage, the pipeline has all the information required to analyze a commit. Tree-sitter is then used to parse the source files, while yyjson is used to generate the final dataset entry.

For each affected file, Tree-sitter is used to extract the functions contained in both the previous and current versions of the file. The two resulting lists of functions are then compared in order to identify the changes introduced by the commit. Functions are matched using their name and number of parameters. When a function with the same name and parameter count is found in both versions, it is considered to be the same function and is classified as modified. If a function is only present in the previous version, it is classified as removed. Conversely, if it is only present in the current version, it is classified as added.

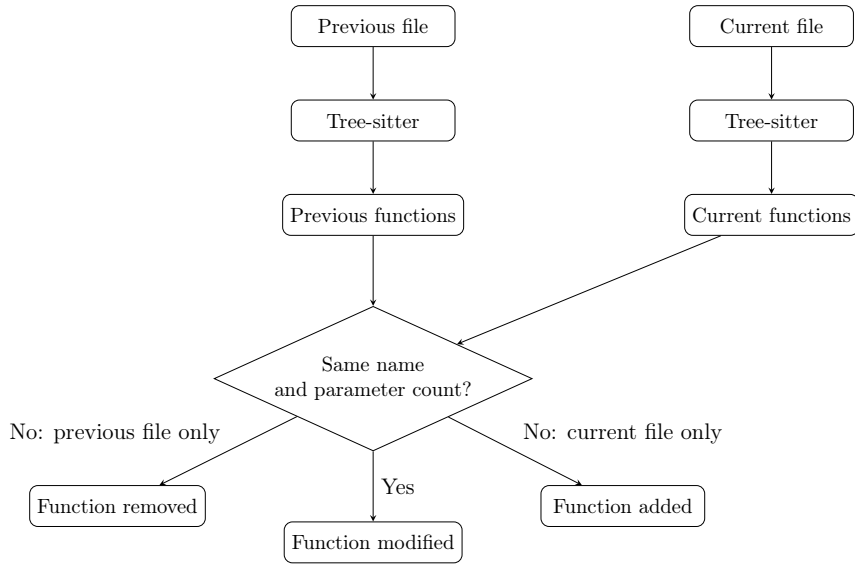


Figure 6: Function comparison between previous and current file versions.

Once all affected files have been processed, the extracted information is combined with the vulnerability and commit metadata collected during the previous stages. This includes information such as the CWE, CVE, repository, commit, affected files, identified functions, and, when enabled, the selected context files. Finally, the complete information associated with the commit is exported as a JSON file using yyjson. Each processed GitHub commit therefore corresponds to one entry in the generated dataset, stored in a separate JSON file.

2.5 Context Extraction

The affected files alone may not always provide enough information for further vulnerability analysis. The pipeline can therefore retrieve additional files from the repository and include them as context. Context files are selected according to their location in the repository directory structure. A distance is assigned to each context file based on its relative position to the affected files. The user can control how far from the affected files the pipeline searches by configuring the context depth.

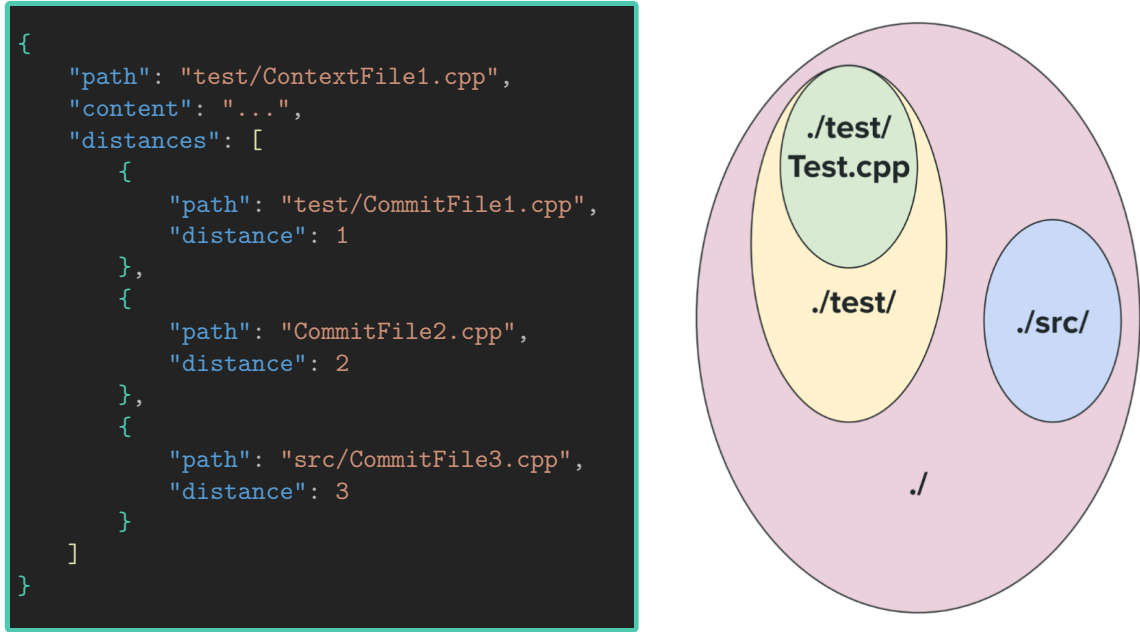


Figure 7: Example of repository context extraction and associated distances.

Figure 7 illustrates an example of the generated context information. The JSON entry contains the path and content of a context file, together with the distances calculated relative to the affected files. The directory representation on the right illustrates how these distances are determined from the repository structure.

The number of context files is also limited by the configuration in order to avoid retrieving excessive amounts of data from large repositories. Context extraction is optional and can be disabled when only the affected files are required.

2.6 Configuration

The pipeline was designed to be configurable so that the user can adapt the data extraction process to their needs without modifying the source code. The configuration file mainly controls the criteria used to retrieve vulnerabilities from the NVD and the constraints applied when processing GitHub repositories.

For the NVD, the user can specify the CWE categories and the date range of the vulnerabilities to retrieve. The configuration also contains the API keys required to access the external services.

For the GitHub processing stage, several parameters can be adjusted, such as the supported source languages (C and/or C++), the context depth used to select additional files, and the maximum number of files that can be processed for a commit. These parameters make it possible to control the amount of data retrieved and the level of repository context included in the generated datasets.

All these parameters are defined in a single configuration file, allowing different datasets to be generated by changing the extraction criteria without recompiling the application. The full configuration file is available in the project repository [README](#).

2.7 Pipeline Architecture

The three processing stages described previously are connected through a simple parallel architecture. The main goal of this architecture is to allow the retrieval and analysis of different commits to progress concurrently, while keeping the implementation relatively simple.

The pipeline uses three threads in total. The main thread is responsible for querying the NVD and creating jobs containing the information required for the GitHub processing stage. These jobs are placed in a queue and processed by a dedicated GitHub worker. Once a commit has been successfully retrieved and validated, the worker places a new job containing all the information collected so far into a second queue.

A dedicated parsing worker retrieves these jobs and performs the source-code analysis described in the previous section. It adds the extracted functions and other parsing information to the existing job structure before generating the final JSON file.

An important aspect of this design is that each stage does not need to retrieve the information produced by the previous stage again. Instead, the job structure is progressively completed as it moves through the pipeline. The GitHub worker adds the commit and file information, while the parsing worker adds the source-code analysis results. This modular design also makes it relatively easy to adapt the pipeline to additional vulnerability or repository sources.

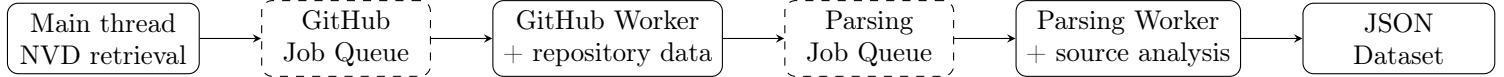


Figure 8: Parallel architecture of the vulnerability extraction pipeline.

The number of workers was also chosen based on practical observations during testing. Adding more GitHub workers did not significantly improve the overall processing time because the network was the main bottleneck. A single parsing worker was generally able to process the jobs produced by the GitHub worker quickly enough. The final architecture therefore uses one GitHub worker and one parsing worker, in addition to the main thread.

2.8 Results

The final result of the pipeline is a structured dataset in which each processed GitHub commit is represented by a separate JSON file. Each entry combines the vulnerability information retrieved from the NVD with the commit information, affected files, extracted functions and, when enabled, the selected repository context.

The complete file contains all the information collected during the previous stages, including the vulnerability, repository and commit metadata, affected files, extracted functions and optional context. This structure allows the different types of information to remain linked to the original vulnerability and commit.

As an example, the following figure shows a JSON file representing the structure of a generated dataset entry. In addition, examples of datasets with the configurations used to generate them were added to the project [repository](#).

```

{
  "CWE": 476,
  "CVE": "CVE-2026-21498",
  "CVE_published": "2026-02-15T10:30:00.000",
  "CVE_description": "Null pointer dereference in example library.",
  "repository": "example/project",
  "commit_hash": "abcdef1234567890...",
  "commit_message": "Fix null pointer dereference",

  "files": [
    {
      "status": "modified",
      "path": "src/modified_file_example.c",
      "previous_path": "src/modified_file_example.c",
      "content": "...",
      "previous_content": "...",

      "affected_functions": [
        {
          "name": "modified_function_example",
          "parameters_number": 2,
          "status": "modified",
          "content": "int modified_function_example(int a, int b) {... }",
          "previous_content": "int modified_function_example(int a, int b) {... }"
        },
        {
          "name": "old_function_example",
          "parameters_number": 1,
          "status": "removed",
          "previous_content": "void old_function_example(int value) {... }"
        }
      ]
    },
    {
      "status": "added",
      "path": "src/added_file_example.c",
      "content": "...",

      "affected_functions": [
        {
          "name": "new_function_example",
          "parameters_number": 1,
          "status": "added",
          "content": "void new_function_example(int value) {... }"
        }
      ]
    }
  ],

  "context_files": [
    {
      "path": "README.md",
      "content": "...",
      "distances": [
        {
          "path": "src/modified_file_example.c",
          "distance": 2
        },
        {
          "path": "src/added_file_example.c",
          "distance": 2
        }
      ]
    }
  ]
}

```

2.9 Validation

No formal unit or integration test suite was developed during the project. This was not part of the initial requirements of the project and was not specifically requested during its development. However, several complementary approaches were used to validate the implementation and improve the reliability of the resulting software.

The pipeline was tested on large amounts of real data. In particular, complete NVD queries covering long time periods were processed, resulting in a large number of GitHub commits being retrieved and analyzed. These executions made it possible to verify that the different stages of the pipeline could operate correctly on a realistic workload and helped identify practical issues related to network communication, repository size, context extraction and processing time. Nonetheless, these tests were not intended as a formal benchmark, but rather as a way of verifying the behaviour of the complete pipeline on a realistic workload.

Also, several tools were used to check the quality and correctness of the C code. Cppcheck was used to identify potential programming errors and suspicious code, while Include-What-You-Use was used to check the dependencies between header files. Valgrind was also used to detect memory-related errors during execution. These tools provided an additional way of checking the implementation during development.

2.10 Known Limitations

The current implementation has some known limitations. First, function matching is based only on the function name and number of parameters. This was chosen because comparing parameter types is difficult to implement reliably with Tree-sitter. As a result, this can lead to incorrect matches in C++ code when several overloaded methods share the same name and number of parameters.

```
{
  "affected_functions": [
    {
      "name": "Convert",
      "parameters_number": 1,
      "status": "modified",
      "content": "std::string Conversion::Convert(int val) ...",
      "previous_content": "std::string Conversion::Convert(int val) ..."
    },
    {
      "name": "Convert",
      "parameters_number": 1,
      "status": "modified",
      "content": "std::string Conversion::Convert(int val) ...",
      "previous_content": "std::string Conversion::Convert(float val) ..."
    }
  ]
}
```

Figure 9: Example of incorrectly matched methods.

Figure 9 shows an example of two overloaded C++ methods. The first “modified” match is correct, while the second is incorrectly classified as “modified” and should actually be classified as “added”.

Also, commits with multiple parents are discarded because the pipeline requires a single previous version to perform the before-and-after comparison. This is a design choice made to keep the comparison unambiguous.

Other constraints are inherent to this type of pipeline rather than limitations of the implementation itself. The dataset generation depends on the availability of the external services and repositories used as data sources, particularly GitHub. Similarly, retrieving additional repository context necessarily increases the amount of data that must be processed. Increasing the context depth therefore results in a trade-off between the amount of information available for further analysis and the processing cost.